M T W T F S S
Page No.: YOUVA
Date:

Assignment 1 : Matrix Operations in C

Aim:- To write a C program to perform basic matrix operations such as addition, subtraction, multiplication and transpose of a matrix using two-dimensional arrays.

Objectives:-
* To understand the concept of two dimensional arrays in C
* To perform arithmetic operations on matrices.
* To implement matrix addition and subtraction
* To implement matrix multiplication using nested loops
* To find the transpose of a matrix.
* To develop programming skills using arrays and loops

Theory:-

A matrix is a rectangular arrangement of elements organized in rows and columns. In C programming, matrices are represented using two-dimensional arrays. Matrix operations are widely used in computer graphics, image processing, machine learning, scientific computing, and egineering applications.

1) Matrix Addition : Condition is both matrices must have the same number of rows and columns.
Formula: `C[i][j] = A[i][j] + B[i][j]`

2) Matrix Subbraction: Condition is both matrices must have the same dimensions.
Formula: `C[i][j] = A[i][j] - B[i][j]`

3) Matrix Multiplication: Condition is no of columns of the first matrix must be equal to the no of rows of the second matrix.
Formula: `C[i][j] = Σ[A[i][k] x B[k][j]]` where k=0 to n-1

M T W T F S S
Page No.:
Date:
YOUVA

4) Transpose of a Matrix: The transpose of a matrix is obtained by interchanging its rows and columns.
Formula: A T [i] [j] = A [j] [i]
These operations are implemented using two-dimensional arrays, nested loops, and conditional statements

FAQ:-

1) Explain matrix addition, subtraction, multiplication, and transpose with suitable examples and state the conditions required for each operation.

Ans Addition & subtraction:
* Condition: Both matrices must have the exact same order (same no of rows and columns).
* (Condition: Both matrices must have the exact same order (same no of rows and columns).
* You add or subtract corresponding elements.
* Example :- $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 3 & 5 \\ 8 & 9 \end{bmatrix}$

$A + B = \begin{bmatrix} 4 & 7 \\ 11 & 13 \end{bmatrix}$ $A - B = \begin{bmatrix} -2 & -3 \\ -5 & -5 \end{bmatrix}$

Multiplication:
* No of columns in first matrix equal to no of rows in second matrix.
* Multiply matching rows of first matrix by columns of second matrix & add products together.
* Example:- $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ (2x2) $B = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$ (2 x 1)

$A \times B = \begin{bmatrix} (1 \times 5) + (2 \times 6) \\ (3 \times 5) + (4 \times 6) \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$

Transpose

M T W T F S S
Page No.:
Date:
YOUVA

* No condition
* Swap rows and columns. Rows become columns and columns become rows.
* Example: $A = \begin{bmatrix} 1 & 3 & 5 \\ 7 & 3 & 8 \end{bmatrix}_{(2\times3)}$ $A^T = \begin{bmatrix} 1 & 7 \\ 3 & 3 \\ 5 & 8 \end{bmatrix}_{(3\times2)}$

2) Describe how two dimensional arrays are used to represent matrices in C programing. Explain the process of accessing and manipulating matrix elements using nested loops.

Ans * A 2-D array is essentially a grid organized by rows and columns, declared like, int matrix [rows] [colums];
* Elements are touched using two indices, matrix [row index] [column_matrix]. The first item will be [0] [0].
* To read or update,
  ① The outer loop selects a row.
  ② The inner loop steps across every column in that row.
* Eg:- for (int i=0 ; i < rows ; i++)
  { for (int j=0 ; j < colums ; j++)
  { printf ("%lod", matrix [i] [j]); }

3 Write the algorithm for performing matrix multiplication and explain why compatibility between matrices is necessary before multiplication.

Ans. Compatibility between matrices is necessary because, if matrix A has 3 columns, its row has 3 elements and for multiplication to work matrix B must have exactly 3 rows, otherwise one element will be left without a partner.

Algorithm
① Start
② Check if columns − A == rows B. If no stop
③ Set up a loop for each row i of matrix A

④ Set up a column loop for each column j of matrix B.
⑤ Initialize Result[i][j] = 0
⑥ Run a third loop k from 0 to columns of A - 1
Result[i][j] = Result[i][j] + (A[i][k] * B[k][j])
⑦ End all loops
⑧ Stop

4) Discuss the applications of matrix operations in real-world domains such as computer graphics, image processing, machine learning, egineering and scientific computing.

Ans * Computer graphics: 3-D video games use matrices to scale, rotate and move characters and objects smoothly across your screen.
* Image processing: Digital filters work by multiplying a matrix of image pixels by a small transformation matrix.
* Machine Learning: AI profiles data as massive matrices. Training a neural network relies heavily on multiplying these datasets together.
* Egineering & Scientific Computing: Engineers use matrices to simulate physics, such as calculating structural stress on bridges or forcasting weather patterns using fluid dynamics.

5) Explain the role of nested loops in implementing matrix operations in C. Illustrate how nested loops are used for addition, multiplication and transpose of matrices.

Ans Nested loops mimics the multi-dimensional structure of a matrix. The outer loop moves vertically down the rows, while the inner loop scans horizontally across the columns.

Addition (2 loops)

```
for (int i=0; i<rows; i++)
{   for (int j=0; j<columns; j++)
    { k[C[i][j] = A[i][j] + B[i][j];
```

Transpose (2 loops)

```
for (int i=0 ; i < rows; i++)
{ for (int j=0 ; j < cols; j++)
    { T[j][i] = A[i][j]; }
```

Multiplication (3 loops)

```
for (int i=0 ; i<rows; i++)
{ for (int j=0 ; j<cols; j++)
    { C[i][j]=0;
      for (int k=0; k<cols_A; k++)
      { C[i][j] = C[i][j] + (A[i][k] * B[k][j]; }
    }
}
```

Algorithms:-

1) Main Program Algorithm:

① Start
② Define MAX = 10 for maximum matrix capacity
③ Describe matrices A, B and Result alongside variables, r1, c1, r2, c2 and user's menu choice.
④ Display menu operations
⑤ Read user's choice
⑥ Evaluate choice using a switch-case statement:
* (case 1 (Addition)
  * Input rows and colums for matrix-A (r1, c1) and matrix-B (r2, c2)
  * If r1 ≠ r2 or c1 ≠ c2, print "Error and jump back to step-④
  * Otherwise, input A & B and call Addition function display Result and break

* Case 2 (Subtraction)
  * Input r1, c1, r2, & c2
  * If r1 ≠ r2 or c1 ≠ c2, print "Error" and jump back to step ④.
  * Otherwise, input A & B and call subtraction fn; display Result, break

* Case 3 (Multiplication)
  * Input r1, c1, r2, c2
  * If c1 ≠ r2, print "Error" and jump back to step-④
  * Otherwise, input A & B call Multiplication fn; display Result & break

* Case 4 (Transpose)
  * Input r1 & c1
  * Call Transpose fn; display Result & break

* Case 5 (Exit)
  * Terminate program execution

* Default
  * Print "Invalid Choice" and return to step -④.

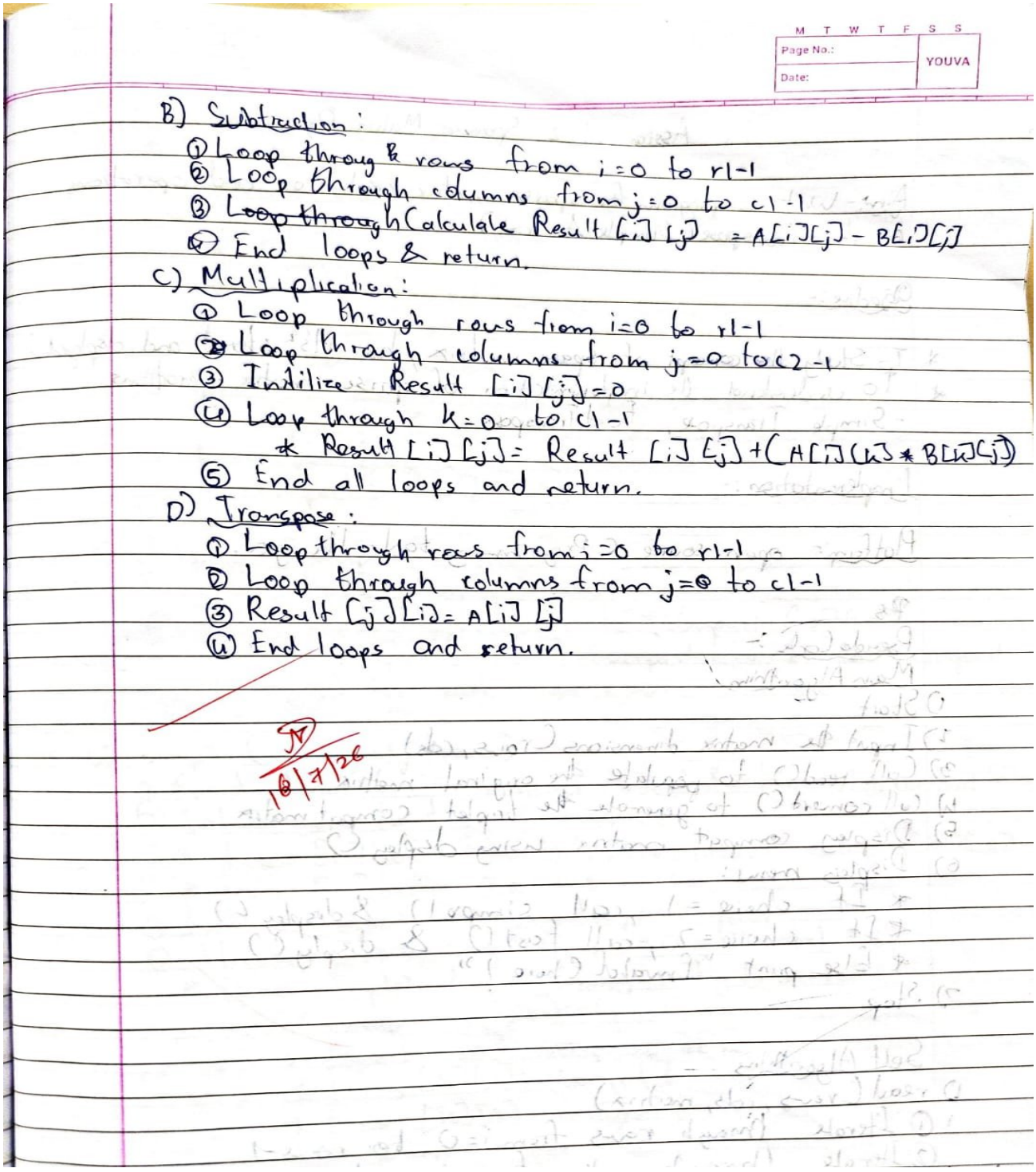
⑦ Stop

2)

2) Sub Algorithims:

A) Addition:

① Loop through raws from i=0 to r1-1
② Loop through colums from j=0 to c1-1
③ Calculate Result [i][j] = A[i][j] + B[i][j]
④ End loops & return.



include <stdio.h>

#define MAX 10

void input (int matrix[MAX][MAX], int rows, int cols);

void display (int matrix[MAX][MAX], int rows, int cols);

```
void add (int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int rows, int cols);

void subtract (int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int rows, int cols);

void multiply (int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int r1, int c1, int c2);

void transpose (int matrix[MAX][MAX], int result[MAX][MAX], int rows, int cols);


int main()

{

    int a[MAX][MAX], b[MAX][MAX], result[MAX][MAX];

    int r1, c1, r2, c2;

    int choice;


    while (1)

    {

        printf("\n--- Matrix Operations Menu ---\n");

        printf("1. Addition of two matrices\n");

        printf("2. Subtraction of two matrices\n");

        printf("3. Multiplication of two matrices\n");

        printf("4. Transpose of a matrix\n");

        printf("5. Exit\n");

        printf("Enter your choice (1-5): ");

        scanf("%d", &choice);


        switch (choice)

          {

           case 1:

               printf("Enter rows and columns for Matrix A: ");

               scanf("%d %d", &r1, &c1);
```

```
    printf("Enter rows and columns for Matrix B: ");
    scanf("%d %d", &r2, &c2);
    if (r1!=r2||c1!=c2)
        {
        printf("\nError! Matrices must have the same order.\n");
        break;
    }
    printf("\nEnter elements of Matrix A:\n");
    input(a, r1, c1);
    printf("\nEnter elements of Matrix B:\n");
    input(b, r2, c2);
    add(a, b, result, r1, c1);
    printf("\nResultant Matrix (A + B):\n");
    display(result, r1, c1);
    break;

case 2:
    printf("Enter rows and columns for Matrix A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns for Matrix B: ");
    scanf("%d %d", &r2, &c2);
    if (r1 != r2 || c1 != c2)
        {
        printf("\nError! Matrices must have the same order.\n");
        break;
    }
    printf("\nEnter elements of Matrix A:\n");
    input(a, r1, c1);
    printf("\nEnter elements of Matrix B:\n");
```

```
    input(b, r2, c2);
    subtract(a, b, result, r1, c1);
    printf("\nResultant Matrix (A - B):\n");
    display(result, r1, c1);
    break;

case 3:
    printf("Enter rows and columns for Matrix A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns for Matrix B: ");
    scanf("%d %d", &r2, &c2);
    if (c1 != r2)
        {
       printf("\nError! Columns of Matrix A must equal rows of Matrix B.\n");
       break;
    }
    printf("\nEnter elements of Matrix A:\n");
    input(a, r1, c1);
    printf("\nEnter elements of Matrix B:\n");
    input(b, r2, c2);
    multiply(a, b, result, r1, c1, c2);
    printf("\nResultant Matrix (A * B):\n");
    display(result, r1, c2);
    break;

case 4:
    printf("Enter rows and columns for the matrix: ");
    scanf("%d %d", &r1, &c1);
    printf("\nEnter elements of the matrix:\n");
```

```
            input(a, r1, c1);
            transpose(a, result, r1, c1);
            printf("\nTransposed Matrix:\n");
            display(result, c1, r1);
            break;

        case 5:
            printf("Exiting program!\n");
                return 0;

        default:
            printf("Invalid choice!\n");
                break;
      }
   }
   return 0;
}

void input(int matrix[MAX][MAX], int rows, int cols)
{
   for (int i=0; i<rows; i++)
   {
      for (int j=0; j<cols; j++)
        {
         printf("Enter element [%d][%d]: ", i, j);
         scanf("%d", &matrix[i][j]);
      }
   }
}
```

```
void display(int matrix[MAX][MAX], int rows, int cols)
{
    for (int i=0; i<rows; i++)
    {
        for (int j=0; j<cols; j++)
          {
           printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }
}

void add(int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int rows, int cols)
{
    for (int i=0; i<rows; i++)
    {
        for (int j=0; j<cols; j++)
          {
           result[i][j] = a[i][j] + b[i][j];
        }
    }
}

void subtract(int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int rows, int
cols)
{
    for (int i=0; i<rows; i++)
    {
```

```
        for (int j=0; j<cols; j++)
          {
           result[i][j] = a[i][j] - b[i][j];
        }
    }
}

void multiply(int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int r1, int c1, int c2)
{
    for (int i=0; i<r1; i++)
    {
        for (int j=0; j<c2; j++)
          {
           result[i][j] = 0;
           for (int k=0; k<c1; k++)
             {
              result[i][j] += a[i][k] * b[k][j];
           }
        }
    }
}

void transpose(int matrix[MAX][MAX], int result[MAX][MAX], int rows, int cols)
{
    for (int i=0; i<rows; i++)
    {
        for (int j=0; j<cols; j++)
          {
```

```
            result[j][i] = matrix[i][j];
        }
    }
}
```

Output:

Enter number of rows and columns: 3 3

Enter matrix elements:

0 0 5

0 8 0

3 0 0

Compact Form:

Row	Col	Value
3	3	3
0	2	5
1	1	8
2	0	3

Choose Transpose Method:

1. Simple Transpose

2. Fast Transpose

Enter your choice (1 or 2): 2

Result:

Row	Col	Value
3	3	3
0	2	3
1	1	8

2 0 5